

An Explainable Multimodal Neural Framework for Financial Risk Management

Ibrahim Hussain

Department of AI & Data Science
FAST NUCES
Islamabad, Pakistan
ibrahimbeaconarion@gmail.com

Lubabah Moten

Department of AI & Data Science
FAST NUCES
Islamabad, Pakistan
lubabahmoten@gmail.com

Sabeel Nadeem

Department of AI & Data Science
FAST NUCES
Islamabad, Pakistan
sabeelnadeem15@gmail.com

Abstract—Financial markets are noisy and unpredictable, yet most AI models operate as black boxes that cannot explain their decisions. This lack of transparency makes them unreliable for financial risk management. This paper proposes an explainable multimodal neural framework that breaks financial risk management into specialised modules. The final system uses four active data streams: market time series, SEC text filings, FRED macro/regime indicators and cross-asset graph structures. A temporal attention encoder learns from market sequences, while FinBERT processes financial text for sentiment and news analysis. These signals feed into a risk engine that combines volatility, drawdown, VaR, CVaR, StemGNN-based contagion risk, liquidity risk and MTGNN-based regime detection. The regime module combines temporal embeddings, FinBERT text embeddings, learned graph properties, and FRED macro features to classify market state. A position sizing engine determines exposure, and an xAI layer provides explanations for every decision. Results show that the framework produces risk-aware Buy/Hold/Sell decisions with confidence scores, position sizes, risk summaries and human-readable explanations. Our work is accessible at <https://github.com/ib-hussain/fin-glassbox>.

I. INTRODUCTION

Modern AI agents and frameworks carry a significant loss of transparency, hiding the inner workings of their models. They act autonomously and make impressively accurate predictions, but there is no accountability. When a model gets something wrong, it can not be held responsible for it, and clients who rely on these models can suffer real losses. This problem is especially critical in finance. Financial markets are noisy, high-risk and constantly changing. The thing about Black box AI is that it does not work well for this domain, even if it is highly accurate. A good prediction is useless if no one can explain why a trade was made or why a portfolio was put at risk. It is difficult to have a single model to learn all the different factors driving the market, such as price patterns, sentiment, risk, contagion and market regime. So when it makes a mistake, debugging it becomes nearly impossible.

II. LITERATURE REVIEW

Over the past years, many modern deep learning methods have been used in finance for stock prediction. “Among the graph modelling technologies, graph neural network (GNN) models can handle the complex graph structure and achieve great performance and thus could be used to solve financial

tasks.”[1] Almost none have built multi-agent systems to model trading decisions. With the excellent results modern models produce, the lack of transparency holds them back from widespread adoption due to their failure to comply with regulations.

Arsenault et al.[2] surveyed xAI techniques applied to financial time series forecasting. They clarified that interpretability and explainability are not the same thing. Interpretable models are transparent by design, like a linear model, where you can see what each feature does, whereas explainable models are black boxes that need SHAP and LIME to explain themselves. They cover linear regressions, decision trees, attention mechanisms and graph-based models for interpretable models and for explainable methods. They cover propagation techniques that track features through the network and techniques like SHAP[7] and LIME[8] that change inputs to see what affects the output. They also highlighted that evaluating how good an explanation is remains an open problem, as there is no standard metric for measuring the reliability of an explanation.

Uygun and Sefer[3] evaluated three GNNs: MTGNN[5], StemGNN[6], and FourierGNN - across two markets: cryptocurrency and forex. What they discovered was that older models like LSTM, and ARIMA fall apart when the data gets messy, but GNNs keep working. They identify patterns, remain accurate, and capture complex relationships among assets. MTGNN[5] learns a graph structure from the data using attention and combines it with temporal convolutions. StemGNN[6] works in the spectral domain to capture spatial and temporal dependencies together. FourierGNN turns every value into a graph node and processes everything in the frequency domain. But the drawback is that these models only predict prices. They do not tell you how uncertain they are. They do not assess risk. They do not explain their decisions.

Xiao et al.[4] proposed TradingAgents, a framework where multiple LLMs collaborate in a structured workflow to simulate a digital hedge fund. Each agent performs a specific human’s job. Different data goes to different agents, so the system gets the full picture. As a result, that context gives them strong predictions. The framework has fundamental analysts studying financials, sentiment analysts tracking social media, news analysts watching macroeconomic events and technical analysts calculating indicators like RSI and MACD. The

framework stands out for its multimodal data and distributed design, where each agent has exactly one job. As well as this, LLMs have general knowledge but not really domain-specific training. They are hard to reproduce, and they do not really learn numerical patterns the way a trained neural network does. They talk their way through decisions, which is flexible but not rigorous enough.

These works show that financial AI benefits from graph modelling, multi-agent structure and explainability. However, existing systems usually focus on either prediction, agent reasoning or post-hoc explanations separately. They rarely combine multimodal encoders, specialist neural analysts, classical risk measures, graph-based contagion, regime detection, position sizing and xAI in one auditable framework. This gap motivates our proposed architecture.

III. PROBLEM STATEMENT

Modern AI agents and frameworks carry a significant loss of transparency, hiding the inner workings of their models. They act autonomously and make impressively accurate predictions, but there is no accountability. When a model gets something wrong, it can not be held responsible for it, and clients who rely on these models can suffer real losses. This problem is especially critical in finance. Financial markets are noisy, high-risk, and constantly changing. The thing about Black box AI is that it does not work well for this domain, even if it is highly accurate. A good prediction is useless if no one can explain why a trade was made or why a portfolio was put at risk. Our system solves this transparency problem by using specialised modules and explainable AI to explain every decision.

A. Input Modalities

The system takes five types of input: market time series data, SEC text filings, macro and regime indicators, company fundamentals and cross-asset graph structures.

B. Output & Task Type

The system produces a Buy/Hold/Sell decision, confidence score, position size, risk summary and explanation report. This is a combination of classification, risk scoring and explainable decision support.

C. Prediction Objectives

This study is guided by the following research objectives:

- 1) **Accurate and risk-aware prediction**
- 2) **Full explainability for every decision**

D. Explainability Objective

We want to ensure that every decision the system makes is easy to follow. The user should be able to see why a certain decision is made, what risks were considered and why the position size was chosen. Basically, the goal is to turn a black box into a glass box so every step can be inspected and trusted.

IV. DATA COLLECTION & PROCESSING METHODOLOGY

We wished to provide a proper picture of the stock market to our models. The picture we wanted to provide needed to contain as many drivers of a company's growth/decline as possible. Only with a proper perspective on the market can accurate predictions be generated.

A. Tabular Fundamentals Data

The first structured data family comprised company-level fundamentals, including revenue, net income, assets, liabilities, equity, margins, leverage indicators, cash flow, and derived profitability ratios. This data was intended to represent firms' internal financial conditions. The main source was the SEC EDGAR¹ Through this database, we obtained two archives: the `companyfacts.zip`, `submissions.zip`.

The initial list contained 851,569 companies registered as U.S. companies. Out of our data, we were able to extract 5,644 companies as the only publicly traded U.S. companies. Through a mess of these files, we managed to obtain cleaned tabular data. This contained point-in-time quarterly fundamentals and derived features, including a clean CIK.² The comprehensive list of 5,644 companies is what became our filter for the rest of the data, although this number was reduced, and the whole data stream of fundamentals data was later removed.

B. Financial Text Dataset

The second data family consisted of textual SEC filings. This data was required for the text stream of the framework, which relied on company news. SEC EDGAR provides official company filings and disclosure documents. They contained management discussion, risk factors, business descriptions, governance disclosures, legal events, liquidity warnings, and other company information. The SEC text pipeline focused only on four raw filing documents:

- **10-K:** "The Form 10-K(Annual Report) is a comprehensive annual summary of a company's performance." [22]
- **10-Q:** "The Form 10-Q(Quarterly Report) is a streamlined version of the 10-K, providing a "snapshot" of the company's financial position during the year." [23]
- **8-K:** The Form 8-K(Current Report) is used to announce significant events that could affect the stock price between periodic reports. [24]
- **DEF-14A:** "The DEF 14A(Definitive Proxy Statement) is filed before a shareholder meeting." [25]

The text engineering script contained stages for inventory, cleaning, section extraction, quality reporting, dataset balancing, and chunk generation for encoding. Eventually, around 288k raw filings were collected. A major issue occurred when

¹"Electronic Data Gathering, Analysis, and Retrieval(EDGAR) database used by the U.S. Securities and Exchange Commission (SEC) to store and provide access to documents filed by public companies and other entities." [15]

²CIK(Central Index Key) is a unique, permanent identification number assigned by the SEC to individuals, companies, and foreign governments. It is used to identify filings within the EDGAR database, ensuring all regulatory submissions are correctly associated with the filer.

the pipeline reported only 23 usable years under the balancing rule. A targeted top-up script was therefore created to detect weak years and download additional manifests and balance the whole dataset with regard to them.

C. Macro & Regime Data

This data was required to capture the wider economic environment in which market movements occur and the conditions surrounding it in regard to other economic factors. It included interest rates, inflation indicators, labour-market variables, credit spreads, volatility proxies, recession indicators, exchange rates, and other stress/regime markers.

The primary source was FRED(Federal Reserve Economic Data).³[21] Initially, it was obtained through raw acquisition, transformation, daily alignment, and missingness reporting. The pipeline evolved in stages. The workflow contained filtering more than 11,000 series down to interpretable series with sufficient 2000–2024 coverage. The final cleaned macro/regime dataset contained 49 series and no missing values after cleaning.

D. Market Data Collection & Cleaning

It is the main pillar for regime modelling, cross-asset graph construction, as well as for the usage of most of the models of our framework. The required data consisted of:

- Open, high, low, close, and volume values.
- Adjusted prices(where available).
- Daily returns & log returns.
- Trading volume & liquidity features.
- Technical indicators such as RSI, MACD, Bollinger position, volatility, and price-range features.
- Benchmark-relative features & SPY correlation.

The market data initially had only 61% of the data for 4,428 tickers. So the pipeline had to be expanded and completed using additional sources.[17][18][19][20] The first technical issue was that different sources used different price conventions. To solve this, overlapping dates were used to compute a per-ticker median scaling ratio. This adjusted scale data was used to fill any missing data cells in the main dataset. After comprehensive merging, we were even able to get 62% data coverage. So we decided to prune the number of tickers to the top 2500 tickers with the most coverage. The constraint of data unavailability at this point was caused by the IPO date or the delisting of a company. So to have balance and availability throughout the data, we turned to synthetic data generation for filling missing values. Now the data was divided into 3 categories. The first category was tickers with at least 6,000 days of coverage. These were filled using linear imputation. The second category was the tickers with at least 50% coverage; these stocks were mostly the ones that had a late IPO, so their data wasn't really available for the past.

³“FRED is an online database consisting of hundreds of thousands of economic data time series from scores of national, international, public, and private sources. FRED, created and maintained by the Research Department at the Federal Reserve Bank of St. Louis, goes far beyond simply providing data.”[16]

For these tickers with backward data missing, the data was mirrored where backward filling was needed, hence reversing all trends. Then the auto-regressive property of ARIMA was used to fill data, resulting in around 10-15% of the data left. The third method was KNN imputation applied on tickers with less than 50% coverage. This used tickers of similar sector for producing the values of the stocks of that sector. With this 3-step pipeline, we generated synthetic data for our use. The usage and generation of synthetic data was justified due to the fact that when the generated data was compared with the organic data, the spreads and distributions of the features of the data were almost perfect matches. Hence, it was concluded that it was sensible to use this data for further usage.

E. Cross-Asset Relation Graphs

The final data family was the cross-asset relation dataset. Unlike the other data families, this was derived from the final market-data panel. This data was required because financial assets do not move independently. A single-stock model can only analyse one company at a time, whereas a graph-based model can represent how shocks, correlations, sector movements, ETF-like flows, and market-wide stress propagate across assets. The cross-asset relation specification defined several graph types, including correlation networks, sector graphs, ETF-style relationships, index membership graphs, and relationship vectors. The graph pipeline used derived alternatives:

- SIC-code-based sector mapping instead of yfinance sector metadata.
- ETF return correlation instead of live ETF holdings.
- Median dollar volume as a market-cap proxy.
- Beta versus SPY from returns.
- Rolling return correlations for dynamic graph snapshots.

The final graph pipeline produced 313 rolling correlation snapshots, 8,578 static edges, 2,515 graph nodes, and a sample NetworkX graph with 105,545 edges. It also had full coverage for sector mapping, market-cap proxy values, and beta estimates.

Our final analysis showed that the overlap between the final market universe and the fundamentals dataset was limited. This reduced the usefulness of the fundamentals stream for final modelling. So the fundamentals data stream was fully removed.

V. PROPOSED FRAMEWORK

The proposed framework is a neural system. The entire system can be divided into 4 layers. The system works on 512-dimensional embeddings as inputs. Each module is supposed to take its inputs from its upstream module(s), perform the train/test/val on it and convey its outputs to its downstream modules.

A. Encoder Layer

At the top, we have an encoder layer to process an input sequence from our data families and map the entire input into a continuous vector space that captures dependencies,

relationships, and context between data. For the purposes of this work, we use 2 parallel encoders.

1) *Temporal Market Encoder*: The Temporal Encoder encodes attention across time series data. The encoder needs to be trained from scratch. The encoder uses self-supervised masked reconstruction. This allows it to learn return dynamics, volume-price relationships and cross-feature interactions inside a 30-day market window. The encoder uses feature normalisation so that features with large numeric ranges do not dominate. Normalisation must be fitted on the training split only for a chunk to prevent data leakage. Validation and test embeddings must reuse the training-fitted normaliser. They must not fit their own normalisers using validation/test data because that would leak future distribution information into inference. The encoder can explain what shaped the embedding. Temporal Encoder xAI should be interpreted as a representation-level explanation, not a final decision explanation.

2) *Financial Text Encoder*: For this task, we use a fine-tuned version of BERT called FinBERT.[13] FinBERT is used because the input language is financial. SEC filings contain specialised terminology, accounting language, forward-looking statements, management discussion, business sections, and governance text. The FinBERT stage is primarily self-supervised domain adaptation, not supervised market prediction. The training objective is Masked Language Modelling loss. This means the model learns SEC disclosure language better, but it does not directly learn returns, drawdowns, risk classes, or sentiment labels. Those tasks are handled later by downstream trained modules. FinBERT improves the representation of financial text. The base model is ProsusAI/finbert. It has a base embedding dimension of 768, MLM probability of 0.15, and projection dimension of 256. We run domain-adaptive MLM training, which tokenises the training and validation data. This is done for applying dynamic masking, training with mixed precision(where enabled), and recording the training/validation losses. After training, we extract 768-dimensional mean-pooled embeddings from the saved frozen model. Now we fit an IncrementalPCA[14] model on train-only 768-dimensional embeddings, then transform train/validation/test embeddings into 256-dimensional final vectors. The final embeddings use train-only PCA projection.

B. Analyst Layer

1) *News Analyst*: This module is dependent upon FinBERT embeddings & text-market labels. The News Analyst groups FinBERT text-chunk embeddings into document-level units and predicts event impact, importance, risk relevance, volatility-spike likelihood, and drawdown-risk likelihood. The Sentiment Analyst focuses on market-aligned polarity. The News Analyst focuses on event importance, risk relevance, and document-level impact. We construct 3 different labels first:

- News Event Impact
- News Importance
- Risk Relevance

Document chunk embeddings are first projected to a learned representation space ($d_{\text{model}} = 128$). One self-attention layer with 4 heads generates contextualised chunk representations. Multi-head attention pooling aggregates chunk-level information into a unified document representation, with attention weights providing interpretability. An MLP trunk processes the pooled representation, feeding into five prediction heads: event impact, news importance, risk relevance, volatility spike, and drawdown risk. Each of them has a different activation function.

2) *Sentiment Analyst*: The Sentiment Analyst converts FinBERT text embeddings into supervised financial sentiment outputs. It is not a dictionary-based sentiment model, and it is not trained on dummy labels. Its labels are built from future market behaviour after the SEC filing date. In practical terms, it answers:

Given this financial text representation, what market-aligned sentiment signal does it imply?

FinBERT remains the text encoder; the Sentiment Analyst is the task-specific supervised head that maps those text embeddings into sentiment, confidence, uncertainty, and compact downstream sentiment embeddings.

3) *Technical Analyst*: For each embedding date, the module constructs targets using future returns and technical indicators. The labels are: Trend target, Momentum target, and Timing target. The implementation uses robust MACD normalisation with percentile clipping, median, and MAD. This prevents large MACD outliers from collapsing timing targets around 0.5. We use a sequence encoder, a BiLSTM with 1 layer. It reads the embedding sequence to capture forward/backward temporal patterns. Additive attention pooling aggregates timestep information into a single representation, enabling timestep-level explainability.

C. Risk Engine

1) *Contagion Risk Module*: The Contagion Risk Module models how stress can spread from one asset to another. Financial assets do not move independently. If one stock falls sharply, related stocks in the same sector, ETF group or correlation cluster may also become vulnerable. This module, therefore, answers the question:

If market stress propagates through related assets, how vulnerable is each stock over 5, 20 and 60 trading days?

For this module, we use StemGNN[6] because contagion is naturally a graph problem. The input is the cross-asset relation graph built from market-derived relationships such as rolling correlations, sector links, beta estimates and return co-movement. Each stock is treated as a node, and the relationships between stocks are treated as weighted edges. The module predicts contagion risk over three horizons: 5-day, 20-day and 60-day.

The output is not a direct Buy/Hold/Sell decision. It is a risk score that tells the downstream system whether an asset is exposed to wider market stress. This score is passed to

the Position Sizing Engine, Quantitative Analyst and Fusion Engine. If contagion risk is high, the system can reduce exposure even when the individual stock signal looks positive. For explainability, the module exposes important graph edges and the top-influencing neighbouring assets. GNNExplainer[11] can also be used to identify the subgraph most responsible for the contagion prediction.

2) *Drawdown Risk Module*: The Drawdown Risk Module estimates how severe a future downside move can become. It is not the same as volatility. Volatility measures movement in both directions, but drawdown focuses only on the fall from a recent peak to a later low. This matters because a stock can have a positive final return but still suffer a dangerous fall in between.

The module uses Temporal Encoder embeddings and predicts 10-day and 30-day drawdown behaviour. Its outputs include expected drawdown, drawdown risk score, recovery delay and confidence. This module does not decide whether to buy or sell. It only tells the system how painful the downside path may become. If the technical signal is positive but the drawdown risk is high, the downstream system should become more conservative. For xAI, attention weights and gradient-based explanations show which recent timesteps and embedding features contributed most to the drawdown estimate.

3) *Volatility Risk Module*: The Volatility Risk Module estimates how unstable the asset is likely to be over the next short-term and medium-term period. A high volatility forecast does not automatically mean SELL. It means the asset is uncertain, and the system should usually reduce confidence or position size.

This module uses a hybrid design. A GARCH-style[28] baseline and recent realised volatility provide classical financial risk anchors, while a learned MLP uses Temporal Encoder embeddings to adjust the forecast. This gives the module both interpretability and flexibility. The output includes 10-day volatility, 30-day volatility, volatility regime, confidence and volatility risk score. The reason for keeping this module separate is simple: the Technical Analyst searches for opportunity, while the Volatility Module checks whether that opportunity is unstable.

4) *Historical VaR & CVaR Module*: The VaR and CVaR module is kept statistical instead of neural. This is intentional because these risk measures are stronger when they are transparent and directly auditable. Historical VaR estimates the loss threshold at a given confidence level. For example, 95% VaR gives the loss boundary exceeded only in the worst 5% of historical observations.

CVaR, or Expected Shortfall, goes further and measures the average loss beyond the VaR threshold. This is important because VaR only tells where the tail starts, while CVaR tells how bad the tail becomes. The module uses a rolling historical window and produces 95% and 99% VaR/CVaR values. These values are later used by Position Sizing and the Quantitative Analyst to control tail-risk exposure.

5) *Liquidity Risk Module*: The Liquidity Risk Module checks whether a stock can actually be traded safely. A model

may find a good signal, but if the stock has weak volume or high slippage, then the trade is not practical. This module uses dollar volume, volume ratio, turnover proxy, slippage estimate and days-to-liquidate.

The output is a liquidity score and tradability flag. Poor liquidity can reduce the final position or block it completely. This module is rule-based because liquidity constraints must remain clear. If a trade is blocked due to liquidity, the system should explain exactly which rule caused it. This makes the final decision more realistic because it connects prediction with execution feasibility.

6) *Regime Risk Module*: The Regime Risk Module identifies the current state of the market. It classifies the environment into states such as calm, volatile, crisis or rotation. This is important because the same signal can mean different things in different market conditions. A positive technical signal during a calm market is not the same as a positive signal during a crisis.

This module uses multiple data streams. First, it takes Temporal Encoder embeddings, which represent recent market behaviour from price and volume data. Second, it takes FinBERT text embeddings, which represent financial text context from SEC filings. These two embedding streams are used by an MTGNN-style graph builder[5] to learn a feature-aware connectivity graph between stocks. In our framework, MTGNN is not used as a full price forecasting model. It is used only to learn market structure.

After the graph is built, graph-level properties such as density, average degree, edge weight, entropy and graph stress are extracted. This is where the FRED macro/regime features, such as yield spreads, interest-rate indicators, credit spreads and macro stress signals, are combined with the learned graph properties. The classifier then uses both market connectivity and macroeconomic context to detect the regime.

The output includes the regime label, regime confidence, probabilities for each regime class and graph stress values. These outputs are passed to Position Sizing and the Quantitative Analyst. If the system detects a crisis or high-volatility regime, exposure can be reduced even if other modules are optimistic. For xAI, this module explains the decision using graph properties, macro stress contribution and regime probabilities.

7) *Position Sizing Engine*: The Position Sizing Engine decides how much capital should be allocated after all risks are considered. It does not predict market direction. It converts risk into exposure. This is important because BUY, HOLD and SELL are not enough in financial decision-making. A BUY with 2% exposure and a BUY with 10% exposure are very different decisions.

The engine receives volatility, drawdown, VaR/CVaR, contagion, liquidity and regime risk scores. It then calculates a combined risk score and applies hard caps where needed. The module is rule-based so that position sizing remains defensible. Optimistic technical signals cannot override severe risk. The final output is a recommended capital fraction,

capital percentage, binding cap source and explanation of any reduction.

D. Synthesis Layer

After the individual analysts and risk modules produce their outputs, the system needs a way to combine them. The synthesis layer combines the produced outputs of the individual analysts and risk modules by separating the framework into two branches: qualitative analysis and quantitative analysis. This prevents the final model from becoming a messy collection of unrelated signals.

1) *Qualitative Analyst*: The Qualitative Analyst combines text-side evidence from the Sentiment Analyst and News Analyst. The Sentiment and News modules produce structured outputs. The Qualitative Analyst learns how to combine these outputs into a qualitative score, qualitative risk and qualitative confidence.

This module is important because financial text is sparse. Many ticker-days have no filing or text event. In that case, the qualitative branch should stay neutral instead of pretending there is a positive or negative signal. Its role is therefore to summarise available text evidence, not to force a trade. The output is passed to the Fusion Engine as the qualitative branch.

2) *Quantitative Analyst*: The Quantitative Analyst combines the technical signal with all risk-engine outputs. It receives the Position Sizing output, which already contains volatility, drawdown, VaR/CVaR, contagion, liquidity, regime and technical context. The purpose is to create a risk-adjusted numerical view of the market.

The main feature of this module is attention-weighted pooling across risk scores. This means the model can show which risk source mattered most for a ticker-date. For example, one HOLD decision may be caused by contagion risk, while another may be caused by liquidity or regime risk. The output includes quantitative signal, quantitative risk, confidence, branch recommendation and risk-attention weights.

3) *Fusion Engine*: The Fusion Engine combines the qualitative and quantitative branches into the final decision using a hybrid design. The first layer is learned and estimates the branch weights, final signal, final risk and confidence. The second layer is rule-based and acts as a safety barrier.

This is important because the learned model should not be allowed to ignore risk. If the learned layer proposes BUY but liquidity is poor, or crisis risk is high, the rule barrier can reduce the position or change the action to HOLD. The final output includes Buy/Hold/Sell, confidence, position size, branch weights and rule-barrier reasons. This makes the final decision auditable rather than hidden inside one model.

E. Explainability Layer

The xAI layer is the most important part of the framework. Each module produces its own explanation, and the final system combines these explanations into one decision trace.

The system uses module-level and system-level explanations. Neural modules use attention, gradients, counterfactuals, SHAP[7] or LIME[8] where appropriate. Graph modules can

use graph properties and GNNExplainer-style explanations. Rule-based modules, such as liquidity and position sizing, use direct rule traces. The final explanation shows what the system decided, why it decided it, which risks mattered most, whether any rule changed the learned decision, and what position size was finally allowed.

This is the main reason the framework can be called explainable. The final output is not only a prediction. It is a structured decision with risk, confidence, size and a human-readable reason behind it.

VI. EXPERIMENTAL SETUP

A. Implementation Details

For the implementation of our work, we used a machine with above 10,000 CUDA cores & 24GB VRAM. Python 3.12 on Linux was used. We split the data into three chronological chunks spanning 25 years:

- **Chunk 1 (2000-2006)**: training 2000–2004, validation 2005, testing 2006
- **Chunk 2 (2007-2016)**: training 2007–2014, validation 2015, testing 2016.
- **Chunk 3 (2017-2024)**: training 2017–2022, validation 2023, testing 2024.

We distributed the data into pieces for a few reasons. The first chunk included the Dot-com bubble that developed during the late 1990s and peaked on March 10, 2000. After the bubble came gradual economic stability. The second chunk included the 2008 financial crisis, followed by the U.S. government steering the economy to recovery in the following years. The third chunk includes a picture of the market pre-COVID, during COVID, post-COVID, and its effect on global markets, followed by investor optimism in the years after it. These 3 chunks provide a diverse set of data for the model to train on. Using these, the model is evaluated on different time periods separated by long training periods, enough for it to properly understand changes in patterns over time and across thousands of tickers, as well as prevent look-ahead bias with each chunk having its own validation and testing data.

B. Training Protocol

For each chunk, the model was fitted only on the training years, tuned on the validation year, and evaluated once on the held-out test year. This was done to prevent look-ahead bias and to keep the evaluation closer to a real financial setting, where future market information is not available during training.

Before training, all module-specific scalars, normalisers and dimensionality-reduction steps were fitted only on the training split. The same fitted transformations were then reused for validation and test data. For example, Temporal Encoder normalisation and FinBERT PCA projection were not refitted on validation or test splits. This ensured that no future distribution information leaked into the model.

For trainable components, including the encoders, analyst modules, volatility/drawdown models, graph modules, quantitative analyst and fusion layer, hyperparameters were selected

using Tree-structured Parzen Estimator (TPE) Bayesian optimisation. Depending on module complexity, each search used 5-75 trials. The main search parameters included learning rate, dropout, weight decay, hidden size, number of layers, batch size and early-stopping patience. Learning rates were searched on a log scale, typically between 5×10^{-6} and 8×10^{-4} , while dropout ranged from 0.01 to 0.35 and weight decay ranged from 1×10^{-6} to 5×10^{-3} .

During training, validation loss was monitored after each epoch. The best checkpoint was selected using validation performance rather than final training loss. Early stopping was applied with patience values between 5 and 20 epochs, depending on the module.

Non-trainable risk components were treated differently. Historical VaR and CVaR were computed using a 504-trading-day rolling window, while the liquidity module used fixed rule-based formulas based on dollar volume, volume ratio, turnover proxy and slippage estimates.

C. Baselines

As for our baseline papers, we established three separate baselines. The first baseline represents results gathered by authors using Graph Neural Networks (GNNs) for time-series forecasting.[3] In this work, Uygun and sefer applied MTGNN[5], StemGNN[6], and FourierGNN to two dataset cryptocurrency and forex time series. They report a MAPE of 13.30% and an A20 of 0.777 on the cryptocurrency dataset. The second baseline is that of `TradingAgents`, which relies on natural-language reasoning. It is a multi-LLM framework designed to split financial decision-making among a variety of specialised analysts, while it achieved the best cumulative returns and Sharpe ratios of the strategies tested on major stocks for the three months from January 1 through March 21, 2024. It also provides natural language justification associated with numerical values that could be used for attributions. In contrast, the news Analyst has F1 of 0.877-0.0939 with perfect recall, the VaR 95% breach rate of 4.74% 4.74% is well within the range of acceptable for a breach rate of 5.00%, the average liquidity coverage for all tickers is 92.31%, which means that on average, there are 92.31% of the 2,500 tickers that are always tradable, and each decision is fully traceable through the intermediate outputs of the separately trained sub-modules. The intermediate outputs provide the gradient importance scores, attention weights, and edge importance scores per GNNExplainer, as well as a step-by-step trace of the rules that were applied.

D. Evaluation Metrics

We used three categories of metrics:

- **ML Metrics:** Accuracy, Precision, Recall, F1 score, ROC, AUC, MSE
- **Financial and risk metrics:** Volatility error (MSE), breach rate, Drawdown prediction error (MSE), Decision distribution

- **Explainability metrics:** Attention visualisation, GNNExplainer edge importance, Module contribution breakdown, Qualitative case study explanation

For guidance on running the experiment independently for reproduction purposes, we have provided the entire setup instructions.[26]

VII. RESULTS AND DISCUSSION

A. Analyst Layer Performance

1) *News Analyst:* The News Analyst achieved strong risk detection across all three chunks (Table I). Precision drops to 0.781 on Chunk 3 (2024), which means the model flags more documents as risky than it should. This is probably because the filing language shifted after 2020, as well as the low availability of data for training on this time period.

TABLE I
NEWS ANALYST RISK-DETECTION PERFORMANCE

Chunk	Train Accuracy	Train MSE	Test Accuracy	Test MSE
C1	84.79%	0.063	73.06%	0.049
C2	89.43%	0.036	89.43%	0.044
C3	80.59%	0.043	80.59%	0.041

Binary task: high-risk vs. low-risk.

2) *Sentiment Analyst:* The Sentiment Analyst maps text embeddings to bearish, neutral, or bullish signals. As Table II shows, directional accuracy falls from 47.0% on Chunk 1 to 30.0% on Chunk 3, and polarity correlation is near zero. Formal legal disclosures are just not a good source of market direction. We therefore treat this module as a soft signal: it contributes a small weight to the synthesis layer rather than driving decisions on its own.

TABLE II
SENTIMENT ANALYST DIRECTIONAL ACCURACY AND POLARITY ESTIMATION

Chunk	Train		Validate		Test	
	Accuracy	MSE	Accuracy	MSE	Accuracy	MSE
C1	47.13%	0.1196	47.04%	0.0870	46.00%	0.0907
C2	41.28%	0.1100	31.72%	0.0706	44.60%	0.1067
C3	44.36%	0.1153	30.03%	0.1132	31.76%	0.1042

B. Risk Engine Performance

1) *VaR/CVaR Tail Risk Module:* We evaluated the non-parametric VaR/CVaR module (2-year rolling window) across 15.7 million asset-day observations (2,500 tickers, 6,288 days). Table III shows the VaR 95% breach rate came in at 4.74%, close to the expected 5%, so calibration looks good. The CVaR 95% absolute error of 0.0233 is a reasonable tail-loss estimate. The VaR 99% breach rate of 0.00% is an artefact of our proxy return method during evaluation. It is insufficient and needs to be re-evaluated with a full return series.

2) *Liquidity Risk Module:* The rule-based liquidity module ran across the full ticker universe with no missing values. Table IV shows that 92.31% of asset-days are tradable, with a mean slippage estimate of 2.25%. Most positions can be executed, but a meaningful fraction carries slippage risk that affects position sizing.

TABLE III
VAR/CVAR CALIBRATION & TEST-SET ESTIMATES

Overall Calibration Statistics		Value		
Metric				
VaR95 breach rate, exp. 5%		4.74%		
VaR99 breach rate, exp. 1%		0.00%*		
CVaR95 realised tail loss		−0.0419		
CVaR95 predicted		−0.0652		
CVaR95 absolute error		0.0233		
Average Test-Set Estimates by Chunk				
Metric		C1	C2	C3
Avg. VaR95		−2.53%	−4.08%	−4.34%
Avg. CVaR95		−4.11%	−5.86%	−6.45%
CVaR/VaR tail ratio		1.74	1.53	1.49

*No VaR99 breaches observed in the evaluated sample.

TABLE IV
LIQUIDITY RISK MODULE STATISTICS

Metric	Split	C1	C2	C3
Average Score	Training	0.539	0.553	0.569
	Validation	0.572	0.596	0.588
	Testing	0.574	0.562	0.572
Average Slippage	Training	3.14%	2.28%	1.80%
	Validation	2.24%	1.86%	1.08%
	Testing	1.72%	2.09%	1.48%
Tradable	Training	95.6%	95.1%	95.4%
	Validation	95.5%	98.1%	99.4%
	Testing	98.1%	94.2%	96.8%

Score uses dollar-volume, volume-ratio and turnover sub-scores.

3) *Volatility & Drawdown Modules*: These modules represent the market-risk side of the Risk Engine. The Volatility Module estimates how unstable the price movement is over 10-day and 30-day horizons, while the Drawdown Module focuses on downside path risk. This distinction matters because volatility captures movement in both directions, whereas drawdown captures the actual fall from a recent peak.

As shown in Table V, the volatility module performs more consistently on the 30-day horizon than the 10-day horizon in most test and validation cases. This suggests that medium-term instability is easier to learn than very short-term noise. The drawdown module also remains relatively stable across chunks, with the lowest errors appearing in the later chunks. Overall, these modules do not produce trade decisions directly. They act as risk-control signals that reduce confidence and position size when market conditions become unstable.

TABLE V
VOLATILITY & DRAWDOWN MODULE MSE ACROSS SPLITS

Split	Horizon	Volatility MSE			Drawdown MSE		
		C1	C2	C3	C1	C2	C3
Train	10d	0.047	0.160	0.099	0.118	0.065	0.072
	30d	0.030	0.106	0.059			
Validate	10d	0.096	0.088	0.062	0.122	0.075	0.069
	30d	0.060	0.055	0.036			
Test	10d	0.136	0.129	0.059	0.119	0.085	0.070
	30d	0.085	0.075	0.032			

GARCH(1,1): $\hat{\alpha} = 0.08$, $\hat{\beta} = 0.90$, persistence = 0.98.

Drawdown uses temporal attention; top step: $t-29$ for C1, $t = 0$ for C2–C3.

4) *Contagion Risk Module*: The StemGNN-based contagion module predicts contagion probabilities at 3 different horizons. Table VI reports test MSE across chunks. Performance is consistent (MSE 0.41-0.58). Chunk 3 (2024) has the lowest error, maybe because market conditions were more stable or the graph structure was easier to learn. GNNExplainer provides edge-importance explanations (top influencer weights

0.002-0.021), which makes contagion pathways interpretable.

TABLE VI
CONTAGION RISK MODULE HORIZON PREDICTION MSE

Split	Chunk	5d	20d	60d
Train	C1	0.4201	0.4736	0.4611
	C2	0.4759	0.4889	0.4982
	C3	0.4340	0.4532	0.4673
Validate	C1	0.5159	0.5763	0.5462
	C2	0.5229	0.5355	0.5322
	C3	0.4316	0.4507	0.4659
Test	C1	0.5233	0.5844	0.5526
	C2	0.4549	0.4698	0.4844
	C3	0.4120	0.4340	0.4496

Top test tickers: C1: ACNB, CRDF, ADAM; C2: AA, AAL, EMO; C3: RENX, ASTC, NTRP.
GNNExplainer edge importance range: 0.53–0.73.

C. Fusion Output Analysis

The Fusion Engine combines the quantitative and qualitative branches using a learned MLP followed by rule-based safety constraints. The learned layer estimates branch weights, fused signal, fused risk and confidence, while the rule barrier prevents the final decision from exceeding the Position Sizing recommendation or violating hard risk constraints.

Table VII shows the final decision distribution on validation and test sets. HOLD dominates across all chunks, ranging from 86.1% to nearly 100% on the test splits. This behaviour is expected because the framework is risk-aware and conservative by design. The system does not treat a weak positive signal as enough for BUY. It requires a strong quantitative signal and acceptable risk conditions.

The learned quantitative branch weight is almost 100% in every chunk. This means the Fusion Engine relied mainly on the dense market and risk information produced by the Quantitative Analyst. The qualitative branch had little influence because text evidence is sparse and often unavailable for many ticker-date rows. Rule barriers changed fewer than 0.1% of raw fusion decisions, which suggests that most risk control was already absorbed before Fusion through Position Sizing and the Quantitative Analyst. Overall, the Fusion layer behaves as a final synthesis and safety layer rather than an aggressive trade generator.

TABLE VII
FUSION LAYER DECISION DISTRIBUTION & BRANCH WEIGHTS

Split	Chunk	HOLD	BUY	SELL	Validation Loss	Quant Branch Weight
Validate	C1	99.9%	0.0%	0.02%	0.000357	99.93%
	C2	87.9%	12.1%	0.05%	0.000634	100.0%
	C3	90.0%	10.0%	≈0%	0.000642	100.0%
Test	C1	100.0%	0.0%	≈0%	0.000357	99.95%
	C2	86.1%	13.8%	0.1%	0.000634	100.0%
	C3	94.3%	5.7%	≈0%	0.000642	100.0%

Rule-based constraints altered fewer than 0.1% of raw fusion decisions.

D. Chunk-Wide Performance

E. Explainability Results

Our framework provides a layer/module-wise xAI for each module. For the temporal, Volatility and Drawdown modules, we show the input gradient importance, showing the

TABLE VIII
CONSOLIDATED MODULE PERFORMANCE BY CHUNK ON TEST SET

Module / Metric	C1	C2	C3
Technical Analyst Acc.	40.00%	46.23%	49.02%
News Analyst F1	0.939	0.925	0.877
Sentiment Analyst Acc.	46.00%	44.60%	31.76%
Quant. Analyst Acc.	35.50%	23.79%	17.31%
Volatility MSE(10d)	0.136	0.129	0.059
Drawdown MSE	0.119	0.085	0.070
Contagion MSE(5d)	0.523	0.455	0.412
Avg. Liquidity Score	0.574	0.562	0.572
Fusion HOLD Rate	100.0%	86.1%	94.3%
VaR95 breach rate	4.74% pooled across chunks		

Metrics differ by module type; higher is better for accuracy, F1 and liquidity score, while lower is better for MSE.

most influential embedding dimensions for each module. The volatility identifies explanations based on GARCH parameters ($\alpha = 0.08$, $\beta = 0.90$, standard deviation of innovations=2.05, time-series persistence=0.98) complemented by gradient xAI. The drawdown modules mainly focus on the most recent observation with temporal attention weights pointing towards $t=0$ in chunk 2 and chunk 3 as opposed to $t=29$ in chunk 1, indicating reliance on recent data points. The regime detection and GNN contagion modules use graph xAI. The macro-features mostly drive the classification decisions, while zero are driven by text features. Counterfactual analysis reveals that the model has a rate of zero per cent in chunks 1 when a stable pre-crisis regime, versus a flip rate of 100 per cent in chunks 2 and 3 during and after the crisis period. The GNN contagion module also identifies the most influential edges and nodes in each era, with top edge importance scores ranging from 0.53 to 0.73, and top influence node weights ranging from 0.002 to 0.21. Different tickers dominate in each era. The Qualitative Analyst primarily identifies risk_relevance, drawdown_risk, and sentiment as the top three features in each chunk, with news_uncertainty highest in chunk 2 due to the 2008 financial crisis. The fusion module identifies that the learned quantitative weight is approximately 99.9-100 per cent across all chunks, which basically means the system learned that the qualitative branch contributed near-zero signal under the space text data, and the rule based were altered less than 0.1% of the time.

VIII. LIMITATIONS & FUTURE WORK

A. Data Constraints

Models trained on pre-2010 filing structures do not transfer well to post-2020 language without periodic retraining. The other reason for this behaviour was data sparsity for post-2020 forms. Due to storage constraints, we were unable to download the full filings database of SEC EDGAR[15], and due to that, the years before 2010 have more coverage. 2022 and 2023 particularly had the least coverage, and the rest of the data needed to be balanced due to their low availability. Although if we were to download the full database, we would surely not face the problem of language transfer as each year after 2000 the amount of forms filed with the SEC has only increased, especially after 2020, so even if there are 4 years after 2020 it is our opinion that there exists enough forms that

could almost balance the language context of the model of post-2020 with that of pre-2020.

B. Modern xAI Methods

The recent work in explainable AI for finance is moving in two direction first is building models that are interpretable by design, which means you can look at them directly and understand what they are doing. For example, attention-based models like AT-LSTM[29], where the models learn to weight which time steps matter the most and graph-based models like KDTCN[30], where the relationships between events are visible in the graph-like structure. On the other direction they are applying explainability methods on top of already existing black-box models after training. SHAP and LIME are currently the most used methods. SHAP assigns a contribution score to every feature for every prediction, telling you exactly how much each input pushed the output up or down. Now LIME works by perturbing the input slightly and fitting a simple model around a single prediction to explain it. Arsenault et al.[2] pointed out that these two concepts interpretability and explainability are two different things. An interpretable model is transparent, while an explainable model is a black box model that needs to explain itself.

C. Future Extensions

There can be many ways this work can be extended in the future for different purposes, perhaps even. Here we propose our insights for future directions:

- 1) Train FinBERT on the full SEC filings data for better results.
- 2) Arrange for full fundamentals data, and then reintegrate the stream into the workflow.
- 3) Deploy a version with weekly learning so that it can handle moving markets over time. Every week, it will need to run a training cycle on the previous week's data.
- 4) Use reinforcement learning with an attention-based fusion module to tune the attention scores so that it understands what amount of importance needs to be given to which module. For this, a human-in-the-loop validation might also be needed with financial analysts to evaluate the performance and whether the explanations are actually useful.
- 5) Extend xAI methods, add formal xAI evaluation: explanation faithfulness, stability, and contrastive explanations.
- 6) Expand the ticker universe back to above 5,000 using better data sources EODHD[27] as well as decrease the market data intervals from daily to perhaps 5-minute intervals.
- 7) Increase the time coverage from just 25 years to at least 55 years, but that would result in a loss of SEC filings data, so the news needs to be arranged using other means, and it will need to be fed into a different way to the model architecture. This would also require fundamental data to be arranged.

- 8) Apply rigorous data engineering practices and use a long-span market dataset of at least 55 years, incorporating every company that has ever traded publicly, regardless of whether it was later delisted, merged, or went bankrupt. The objective is to process the data and design the model architecture so that it can learn patterns across stocks over long horizons without being distorted by the changing number of companies present in the market over time.

There can be other directions as well, which we may have missed, although these are the ones we primarily think are the most important and would enhance this work even more.

IX. CONCLUSION

This paper presented an explainable multimodal neural framework for financial risk management. The system combines different types of data through specialised encoders, analysts, risk modules, and a final hybrid Fusion Engine. Instead of relying on one black-box model, the framework separates the financial decision process into smaller and more inspectable components.

The results show that the system can produce risk-aware Buy/Hold/Sell decisions while also reporting confidence, position size, dominant risk drivers and explanation traces. The News Analyst showed strong risk-detection performance, the VaR/CVaR module produced a well-calibrated 95% breach rate, and the liquidity module confirmed broad tradability across the ticker universe. The final Fusion Engine behaved conservatively, with HOLD dominating most test cases, which is consistent with the framework's risk-first design rather than a weakness of the system.

The main contribution of this work is not only prediction, but also auditability. Each major module produces intermediate outputs that can be inspected, and the final decision can be traced through quantitative evidence, qualitative evidence, risk constraints, position sizing and rule-barrier effects. This makes the framework more suitable for financial settings where trust, transparency and risk control matter as much as raw model accuracy.

ACKNOWLEDGMENT

We are thankful to our supervisor, Dr Qurat-ul-ain, for her guidance, feedback and encouragement throughout the development of this project. We would also like to acknowledge the open-source research community and the publicly available financial datasets and tools that made this work possible, including SEC EDGAR, FRED, Yahoo Finance, and Stooq.

REFERENCES

- [1] J. Wang, S. Zhang, Y. Xiao, and R. Song, "A Review on Graph Neural Network Methods in Financial Applications," *Journal of Data Science*, vol. 20, no. 2, pp. 111–134, May 2022, doi: 10.6339/22-JDS1047.
- [2] P.-D. Arsenault, S. Wang, and J.-M. Patenaude, "A Survey of Explainable Artificial Intelligence (XAI) in Financial Time Series Forecasting," *ACM Computing Surveys*, vol. 57, pp. 1–37, 2024.
- [3] Y. Uygun and E. Sefer, "Financial Asset Price Prediction with Graph Neural Network-Based Temporal Deep Learning Models," *Neural Computing and Applications*, vol. 37, no. 30, pp. 25445–25471, 2025, doi: 10.1007/s00521-025-11586-8.
- [4] Y. Xiao, E. Sun, D. Luo, and W. Wang, "TradingAgents: Multi-Agents LLM Financial Trading Framework," *arXiv preprint arXiv:2412.20138*, 2024. [Online]. Available: <https://arxiv.org/abs/2412.20138>
- [5] Z. Wu, S. Pan, G. Chen, G. Long, C. Zhang, and P. S. Yu, "Connecting the Dots: Multivariate Time Series Forecasting with Graph Neural Networks," in *Proc. 26th ACM SIGKDD Int. Conf. Knowledge Discovery & Data Mining (KDD)*, 2020, pp. 753–763, doi: 10.1145/3394486.3403118.
- [6] D. Cao, Y. Wang, J. Duan, C. Zhang, X. Zhu, C. Huang, Y. Tong, B. Xu, J. Bai, J. Tong, and Q. Zhang, "Spectral Temporal Graph Neural Network for Multivariate Time-Series Forecasting," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 17766–17778.
- [7] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in *Advances in Neural Information Processing Systems (NIPS)*, vol. 30, 2017, pp. 4765–4774.
- [8] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why Should I Trust You?: Explaining the Predictions of Any Classifier," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery & Data Mining (KDD)*, 2016, pp. 1135–1144, doi: 10.1145/2939672.2939778.
- [9] F. S. Khan, S. S. Mazhar, K. Mazhar, D. A. AlSaleh, and A. Mazhar, "Model-agnostic explainable artificial intelligence methods in finance: a systematic review, recent developments, limitations, challenges and future directions," *Artificial Intelligence Review*, vol. 58, no. 232, May 2025.
- [10] J. Černevičienė and A. Kabašinskas, "Explainable artificial intelligence (XAI) in finance: a systematic literature review," *Artificial Intelligence Review*, vol. 57, no. 216, Jul. 2024.
- [11] R. Ying, D. Bourgeois, J. You, M. Zitnik, and J. Leskovec, "GN-NEExplainer: Generating explanations for graph neural networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019.
- [12] W. J. Yeo, W. Van Der Heever, R. Mao, E. Cambria, R. Satapathy, and G. Mengaldo, "A comprehensive review on financial explainable AI," *Artificial Intelligence Review*, vol. 58, no. 111, Mar. 2025.
- [13] D. Araci, "FinBERT: Financial Sentiment Analysis with Pre-trained Language Models," *arXiv preprint arXiv:1908.10063*, Aug. 2019. [Online]. Available: <https://arxiv.org/abs/1908.10063>
- [14] V. Lippi and G. Ceccarelli, "Incremental Principal Component Analysis: Exact Implementation and Continuity Corrections," in *Proc. 16th Int. Conf. Informatics in Control, Automation and Robotics (ICINCO)*, 2019, pp. 473–480.
- [15] U.S. Securities and Exchange Commission, "EDGAR Full Text Search," SEC.gov. [Online]. Available: <https://efts.sec.gov/LATEST/search-index/> Accessed: May 8, 2026.
- [16] re3data.org, "Federal Reserve Economic Data," *Registry of Research Data Repositories*, r3d100010158. [Online]. Available: <https://www.re3data.org/repository/r3d100010158> Accessed: May 8, 2026.
- [17] Yahoo Inc., "Yahoo Finance," Yahoo Finance. [Online]. Available: <https://finance.yahoo.com> Accessed: May 9, 2026.
- [18] Stooq, "Stooq Historical Data," Stooq. [Online]. Available: <https://stooq.com/db/h> Accessed: May 9, 2026.
- [19] B. Marjanovic, "Huge Stock Market Dataset," *Kaggle*, 2017. [Online]. Available: <https://www.kaggle.com/datasets/borismarjanovic/price-volume-data-for-all-us-stocks-etfs> Accessed: May 9, 2026.
- [20] eren2222, "NASDAQ, NYSE, NYSE A, OTC Daily Stock 1962–2024," *Kaggle*, 2024. [Online]. Available: <https://www.kaggle.com/datasets/eren2222/nasdaq-nyse-nyse-a-otc-daily-stock-1962-2024> Accessed: May 9, 2026.
- [21] Federal Reserve Bank of St. Louis, "Federal Reserve Economic Data (FRED)," FRED Economic Data. [Online]. Available: <https://fred.stlouisfed.org/> Accessed: May 8, 2026.
- [22] U.S. Securities and Exchange Commission, "Form 10-K," SEC.gov. [Online]. Available: <https://www.sec.gov/files/form10-k.pdf> Accessed: May 8, 2026.
- [23] U.S. Securities and Exchange Commission, "Form 10-Q," SEC.gov. [Online]. Available: <https://www.sec.gov/files/form10-q.pdf> Accessed: May 8, 2026.
- [24] U.S. Securities and Exchange Commission, "Form 8-K," SEC.gov. [Online]. Available: <https://www.sec.gov/files/form8-k.pdf> Accessed: May 8, 2026.
- [25] U.S. Securities and Exchange Commission, "DEF 14A – Definitive Proxy Statement (Tabcorp Holdings),"

EDGAR Archives, Aug. 3, 2023. [Online]. Available: <https://www.sec.gov/Archives/edgar/data/1394319/000095017023039153/tcon-20230803.htm> Accessed: May 8, 2026.

- [26] Ibrahim Hussain, "SETUP.md – Project Setup Instructions," *fin-glassbox*, GitHub. [Online]. Available: <https://github.com/ib-hussain/fin-glassbox/blob/main/SETUP.md> Accessed: May 8, 2026.
- [27] Unicorn Data Services, "EODHD: Market Data API for Stocks, ETFs & Forex." EODHD. [Online]. Available: <https://eodhd.com/> Accessed: May 9, 2026.
- [28] T. Bollerslev, "Generalized autoregressive conditional heteroskedasticity," *Journal of Econometrics*, vol. 31, no. 3, pp. 307–327, Apr. 1986, doi: 10.1016/0304-4076(86)90063-1.
- [29] X. Zhang et al., "AT-LSTM: An Attention-based LSTM Model for Financial Time Series Prediction," *IOP Conf. Ser.: Mater. Sci. Eng.*, vol. 569, no. 5, p. 052037, Sep. 2019, doi: 10.1088/1757-899X/569/5/052037.
- [30] J. Deng, X. Liang, J. He, and A. Zhiyuli, "Knowledge-Driven Stock Trend Prediction and Explanation via Temporal Convolutional Network," in *Companion Proceedings of the 2019 World Wide Web Conference (WWW '19)*, 2019, pp. 678–685, doi: 10.1145/3308560.3317701.